

SENA

Servicio Nacional de Aprendizaje

TALLER N° 3

ABSTRACCIÓN

Programación Orientada a Objetos en Java

Pilar de la POO: Abstracción

Lenguaje: Java

Nivel: Básico

Ejercicios: 5 ejercicios prácticos + 1 reto integrador

Metodología: Constructivista (aprender-haciendo)

Modalidad: Clases y métodos main en archivos separados

ELABORADO POR

Carlos Arturo Barrientos Suares

*Ingeniero de Sistemas · Especialista en Desarrollo de Software · Máster en Ingeniería de Software y Sistemas
Informáticos*

Marco conceptual del taller

¿Qué es la abstracción?

Cuando manejas un carro, no necesitas saber **cómo funciona el motor por dentro** para poder conducirlo: solo aprendes qué hace el volante, los pedales y la palanca de cambios. El fabricante te expone **lo esencial** y oculta la complejidad. Eso es abstracción: **identificar las características importantes** de un objeto e ignorar los detalles que no son necesarios.

En POO, la abstracción se traduce en definir "**qué hace**" un objeto sin importar todavía "**cómo lo hace**". Por ejemplo: todos los vehículos pueden *moverse*, pero un carro lo hace con ruedas y un barco con hélices.

¿Cómo se aplica en Java?

La abstracción en Java se logra con dos herramientas:

- **Clases abstractas:** se declaran con *abstract class*. **No se pueden instanciar** (no puedes escribir *new Figura()* directamente). Solo sirven como molde para otras clases.
- **Métodos abstractos:** se declaran con *abstract* y **no tienen cuerpo** (no tienen implementación). Las subclases **están obligadas** a implementarlos, si no el código no compila.

¿Por qué es tan importante?

1. **Modelado claro:** permite representar conceptos generales (Animal, Vehículo, MedioPago) sin dar todos los detalles.
2. **Contratos obligatorios:** el compilador te avisa si una hija olvidó implementar un método esencial.
3. **Reutilización disciplinada:** los métodos comunes se escriben una sola vez; los específicos se delegan a las hijas.
4. **Separa el "qué" del "cómo":** quien usa la clase abstracta no necesita conocer los detalles de cada implementación.
5. **Facilita el trabajo en equipo:** mientras uno usa la clase abstracta, otro implementa las subclases.

REGLA DE ORO

Una clase abstracta **no se puede instanciar**. Solo puedes crear objetos de sus **hijas concretas**. Si una hija no implementa **todos** los métodos abstractos del padre, también debe declararse como abstracta.

Ejercicio 1: Figuras Geométricas Abstractas

SITUACIÓN PROBLEMA

Estás desarrollando un programa educativo para niños de primaria. Toda figura geométrica tiene un **nombre** y debe poder **calcular su área**. Sin embargo, la fórmula del área es **completamente diferente** para cada figura: no existe una fórmula universal que funcione para todas.

Tu tarea: diseñar una clase abstracta **Figura** que defina el "contrato" (todo lo que las figuras deben poder hacer), y las hijas concretas **Circulo** y **Triangulo** que lo implementen.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Antes de codificar, piensa:

1. ¿Tiene sentido tener un objeto *Figura* sin especificar qué figura es? (¿"Dame una figura, no importa cuál"?)
2. Si intentas escribir `new Figura("X")`, ¿qué te dirá el compilador?
3. ¿Qué pasa si creas la clase **Cuadrado** pero olvidas implementar `calcularArea()`? ¿El código compila?

Solución guiada

Fíjate en dos cosas: (a) la palabra **abstract class** antes del nombre de la clase, y (b) el método **abstract** que *no tiene cuerpo* — termina con punto y coma.

Archivo: `Figura.java`

```
public abstract class Figura {
    protected String nombre;

    public Figura(String nombre) {
        this.nombre = nombre;
    }

    // Metodo abstracto: las hijas DEBEN implementarlo
    public abstract double calcularArea();

    // Metodo concreto: lo heredan todas las hijas
    public void mostrar() {
        System.out.println("Figura: " + nombre);
        System.out.println("Area: " + calcularArea());
    }
}
```

Archivo: Circulo.java

```
public class Circulo extends Figura {
    private double radio;

    public Circulo(double radio) {
        super("Circulo");
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```

Archivo: Triangulo.java

```
public class Triangulo extends Figura {
    private double base;
    private double altura;

    public Triangulo(double base, double altura) {
        super("Triangulo");
        this.base = base;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return (base * altura) / 2;
    }
}
```

Archivo: MainFiguras.java

```
public class MainFiguras {
    public static void main(String[] args) {
        // Figura f = new Figura("X"); ERROR: no se puede instanciar

        Circulo c = new Circulo(5);
        Triangulo t = new Triangulo(4, 6);

        c.mostrar();
        System.out.println("---");
        t.mostrar();
    }
}
```

BENEFICIO APLICADO

La clase *Figura* define el **contrato**: toda figura debe saber calcular su área. El método *mostrar()* se escribe **una sola vez** y funciona para cualquier hija porque sabe que existirá un *calcularArea()*. Si mañana alguien crea Pentagono, Trapecio o Hexagono, todos serán compatibles con *mostrar()* automáticamente.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Crea una tercera hija **Rectangulo** con atributos **ancho** y **alto**. Su área es ancho × alto. Añádelo al main y prueba *mostrar()*. **Reto extra**: intenta crear una clase **Cuadrado** que **NO implemente** *calcularArea()*. Observa el error del compilador y anótalo. Luego arréglalo implementando el método.

Ejercicio 2: Nómina con Distintos Tipos de Pago

SITUACIÓN PROBLEMA

Una empresa colombiana contrata dos tipos de empleados, ambos con **nombre** y **documento**:

- **EmpleadoFijo**: tiene un salario mensual fijo.
- **EmpleadoPorHoras**: cobra según **horas trabajadas** × **tarifa por hora**.

El sistema de nómina debe garantizar que **todo empleado** pueda calcular su pago, pero la forma es completamente diferente en cada tipo.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Reflexiona:

1. ¿Por qué es peligroso **no obligar** a cada tipo de empleado a implementar *calcularPago()*?
2. ¿Qué pasaría si un desarrollador crea la clase *EmpleadoFreelance* y olvida implementar el cálculo?
3. ¿En qué se parece este ejercicio al Ejercicio 1?

Solución guiada

Archivo: `Empleado.java`

```
public abstract class Empleado {
    protected String nombre;
    protected String documento;

    public Empleado(String nombre, String documento) {
        this.nombre = nombre;
        this.documento = documento;
    }

    public abstract double calcularPago();

    public void mostrarRecibo() {
        System.out.println("Empleado: " + nombre);
        System.out.println("Documento: " + documento);
        System.out.println("Pago del mes: $" + calcularPago());
    }
}
```

Archivo: EmpleadoFijo.java

```
public class EmpleadoFijo extends Empleado {
    private double salarioMensual;

    public EmpleadoFijo(String nombre, String documento,
                        double salarioMensual) {
        super(nombre, documento);
        this.salarioMensual = salarioMensual;
    }

    @Override
    public double calcularPago() {
        return salarioMensual;
    }
}
```

Archivo: EmpleadoPorHoras.java

```
public class EmpleadoPorHoras extends Empleado {
    private int horasTrabajadas;
    private double tarifaPorHora;

    public EmpleadoPorHoras(String nombre, String documento,
                            int horas, double tarifa) {
        super(nombre, documento);
        this.horasTrabajadas = horas;
        this.tarifaPorHora = tarifa;
    }

    @Override
    public double calcularPago() {
        return horasTrabajadas * tarifaPorHora;
    }
}
```

Archivo: MainNomina.java

```
public class MainNomina {
    public static void main(String[] args) {
        EmpleadoFijo ef = new EmpleadoFijo("Ana Rios", "1111", 2200000);
        EmpleadoPorHoras eh =
            new EmpleadoPorHoras("Luis Mora", "2222", 80, 15000);

        ef.mostrarRecibo();
        System.out.println("---");
        eh.mostrarRecibo();
    }
}
```

BENEFICIO APLICADO

Como *Empleado* es abstracta, **obliga** a cada hija a definir cómo se calcula su pago. Es imposible olvidarse: el compilador no permitirá crear una hija incompleta. Esto previene errores en sistemas críticos como la nómina (donde un error puede significar que un empleado no reciba su sueldo o reciba de más).

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade una tercera hija **EmpleadoFreelance** que reciba una lista de **proyectos terminados** (número entero) y un **pagoPorProyecto**. Su cálculo debe ser $\text{proyectos} \times \text{pagoPorProyecto}$. Añádalo al main. Piensa: ¿qué pasa si un freelance no ha terminado ningún proyecto ese mes?

Ejercicio 3: Electrodomésticos del Hogar

SITUACIÓN PROBLEMA

En tu casa hay distintos electrodomésticos. Todos tienen una **marca**, todos pueden **encender** y **apagar** (el proceso es el mismo), pero cada uno tiene una **función principal** distinta:

- El **Televisor** muestra canales.
- La **Nevera** enfría alimentos.

Diseña una clase abstracta **Electrodomestico** que sepa encender/apagar por sí misma, pero delegue la función principal a cada hija.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Piensa:

1. ¿Puede una clase abstracta tener métodos que **Sí tengan cuerpo**? (Como encender y apagar).
2. ¿Cuál es la diferencia entre un método abstracto y uno concreto dentro de una clase abstracta?
3. ¿Por qué es mejor este diseño que copiar los métodos encender/apagar en cada electrodoméstico?

Solución guiada

Archivo: **Electrodomestico.java**

```
public abstract class Electrodomestico {
    protected String marca;
    protected boolean encendido;

    public Electrodomestico(String marca) {
        this.marca = marca;
        this.encendido = false;
    }

    // Metodos concretos - comunes a todos
    public void encender() {
        encendido = true;
        System.out.println(marca + " esta encendido.");
    }

    public void apagar() {
        encendido = false;
        System.out.println(marca + " esta apagado.");
    }

    // Metodo abstracto - cada hija lo implementa
    public abstract void funcionPrincipal();
}
```

Archivo: Televisor.java

```
public class Televisor extends Electrodomestico {

    public Televisor(String marca) {
        super(marca);
    }

    @Override
    public void funcionPrincipal() {
        if (encendido) {
            System.out.println("Mostrando el canal en pantalla...");
        } else {
            System.out.println("Enciendolo primero.");
        }
    }
}
```

Archivo: Nevera.java

```
public class Nevera extends Electrodomestico {

    public Nevera(String marca) {
        super(marca);
    }

    @Override
    public void funcionPrincipal() {
        if (encendido) {
            System.out.println("Enfriando alimentos a 4 grados C...");
        } else {
            System.out.println("Enciendala primero.");
        }
    }
}
```

Archivo: MainHogar.java

```
public class MainHogar {
    public static void main(String[] args) {
        Televisor tv = new Televisor("LG");
        Nevera nv = new Nevera("Samsung");

        tv.encender();
        tv.funcionPrincipal();

        System.out.println("---");

        nv.funcionPrincipal(); // Sin encender - dara mensaje de error
        nv.encender();
        nv.funcionPrincipal();
    }
}
```

BENEFICIO APLICADO

Los métodos *encender()* y *apagar()* son **comunes** a todos los electrodomésticos y están implementados en el padre. La función principal, que es lo único realmente distinto, se delega a cada hija. Este equilibrio entre **lo común y lo específico** es la esencia de la abstracción.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade una tercera hija **Lavadora**. Su función principal es "Lavando ropa a 40 grados C durante 45 minutos". Además, agrégale un método propio **centrifugar()**. Prueba todos los electrodomésticos en el main.

Ejercicio 4: Medios de Pago de un E-commerce

SITUACIÓN PROBLEMA

Una tienda online colombiana (tipo Mercado Libre) permite pagar de distintas formas. Todo medio de pago tiene un **monto** y debe poder **procesarPago()**, pero cada uno lo hace de forma completamente diferente:

- Una **TarjetaCredito** valida el número y cobra directamente.
- Un **PagoEfectivo** genera un código de recaudo para pagar en Efecty o Baloto.

Diseña una clase abstracta **MedioPago** y las hijas concretas.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Reflexiona:

1. ¿Debería el sistema de la tienda conocer **todos los detalles** de cada medio de pago?
2. Si mañana se agrega PSE, Nequi o Bitcoin, ¿tendrías que modificar el código de la tienda?
3. ¿Cuál es la ventaja de que la tienda solo conozca la clase abstracta *MedioPago*?

Solución guiada

Archivo: **MedioPago.java**

```
public abstract class MedioPago {
    protected double monto;

    public MedioPago(double monto) {
        this.monto = monto;
    }

    public abstract void procesarPago();

    public void mostrarMonto() {
        System.out.println("Monto a pagar: $" + monto);
    }
}
```

Archivo: TarjetaCredito.java

```
public class TarjetaCredito extends MedioPago {
    private String numeroTarjeta;

    public TarjetaCredito(double monto, String numeroTarjeta) {
        super(monto);
        this.numeroTarjeta = numeroTarjeta;
    }

    @Override
    public void procesarPago() {
        System.out.println("Validando tarjeta " + numeroTarjeta + "...");
        System.out.println("Cobrando $" + monto + " a la tarjeta.");
        System.out.println("Pago aprobado.");
    }
}
```

Archivo: PagoEfectivo.java

```
public class PagoEfectivo extends MedioPago {

    public PagoEfectivo(double monto) {
        super(monto);
    }

    @Override
    public void procesarPago() {
        int codigo = (int)(Math.random() * 100000);
        System.out.println("Genere el codigo de recaudo: " + codigo);
        System.out.println("Acerquese a Efecty a pagar $" + monto);
    }
}
```

Archivo: MainTienda.java

```
public class MainTienda {
    public static void main(String[] args) {
        TarjetaCredito t =
            new TarjetaCredito(250000, "4111-2222-3333-4444");
        PagoEfectivo e = new PagoEfectivo(80000);

        t.mostrarMonto();
        t.procesarPago();

        System.out.println("---");

        e.mostrarMonto();
        e.procesarPago();
    }
}
```

BENEFICIO APLICADO

El sistema de la tienda solo necesita conocer la clase abstracta *MedioPago*. No le importa si por dentro es tarjeta, efectivo, PSE o Nequi. Esto permite **agregar nuevos métodos de pago** en el futuro **sin modificar** el código existente. Los sistemas profesionales de e-commerce se diseñan exactamente así.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade una tercera hija **PagoPSE**. Debe recibir el **banco** del usuario (String). Su método *procesarPago()* debe: (1) simular la redirección al banco, (2) esperar la confirmación, (3) mostrar el resultado. Añádalo al main y prueba los tres medios de pago.

Ejercicio 5: Sistema de Notificaciones

SITUACIÓN PROBLEMA

Una aplicación envía notificaciones por distintos canales. Toda notificación tiene un **destinatario** y un **mensaje**, y debe poder **enviar()**. Además, después de cada envío se debe **registrar en el log** del sistema (esto es común a todos).

Los canales:

- **NotificacionEmail**: envía a un correo electrónico.
- **NotificacionSMS**: envía a un número de celular.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Piensa:

1. ¿El registro en el log debería estar en el padre o repetirse en cada hija?
2. Si cada hija tiene que registrar el log, ¿cómo puede hacerlo si el método está en el padre?
3. ¿Cuántas notificaciones podrías agregar sin modificar el código actual?

Solución guiada

Archivo: **Notificacion.java**

```
public abstract class Notificacion {
    protected String destinatario;
    protected String mensaje;

    public Notificacion(String destinatario, String mensaje) {
        this.destinatario = destinatario;
        this.mensaje = mensaje;
    }

    public abstract void enviar();

    // Metodo comun - las hijas lo llaman despues de enviar
    public void registrarEnvio() {
        System.out.println("[LOG] Notificacion enviada a: " + destinatario);
    }
}
```

Archivo: NotificacionEmail.java

```
public class NotificacionEmail extends Notificacion {

    public NotificacionEmail(String correo, String mensaje) {
        super(correo, mensaje);
    }

    @Override
    public void enviar() {
        System.out.println("Enviando EMAIL a " + destinatario);
        System.out.println("Asunto: Notificacion del sistema");
        System.out.println("Mensaje: " + mensaje);
        registrarEnvio(); // Metodo del padre
    }
}
```

Archivo: NotificacionSMS.java

```
public class NotificacionSMS extends Notificacion {

    public NotificacionSMS(String celular, String mensaje) {
        super(celular, mensaje);
    }

    @Override
    public void enviar() {
        System.out.println("Enviando SMS al numero " + destinatario);
        System.out.println("SMS: " + mensaje);
        registrarEnvio();
    }
}
```

Archivo: MainNotificaciones.java

```
public class MainNotificaciones {
    public static void main(String[] args) {
        NotificacionEmail email = new NotificacionEmail(
            "carlos@sena.edu.co",
            "Su matricula fue aprobada.");

        NotificacionSMS sms = new NotificacionSMS(
            "3001234567",
            "Su pedido esta en camino.");

        email.enviar();
        System.out.println("---");
        sms.enviar();
    }
}
```

BENEFICIO APLICADO

El método *registrarEnvio()* está en la clase abstracta porque es común a todas las notificaciones. Solo el envío real se delega a cada hija. Mañana se podría agregar *NotificacionWhatsApp*, *NotificacionPush* o *NotificacionTelegram* con muy poco esfuerzo: **ese es el poder de la abstracción**.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade una tercera hija **NotificacionWhatsApp** que reciba un número de WhatsApp. Su método *enviar()* debe mostrar: "Enviando WhatsApp a [numero]: [mensaje]" y luego llamar a *registrarEnvio()*. Añádalo al main. Piensa: ¿tienes que modificar la clase *Notificacion* para agregar este nuevo tipo?

Reto Integrador: Servicios Públicos

SITUACIÓN PROBLEMA

Una empresa en Cúcuta gestiona el cobro de **servicios públicos**. Todo servicio tiene **número de contrato**, **titular** y un **consumo** del mes. Cada tipo cobra de forma distinta:

- **Agua:** consumo en m³. Cobra \$3.500 por m³, con un cargo fijo de \$8.000.
- **Luz:** consumo en kWh. Cobra \$650 por kWh, con un cargo fijo de \$12.000.
- **Gas:** consumo en m³. Cobra \$2.100 por m³, con un cargo fijo de \$5.000.

Todos deben poder **calcularFactura()** y **emitirRecibo()**.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Antes de resolver:

1. ¿Qué método debe ser abstracto en la clase padre?
2. ¿Qué método puede tener implementación en el padre para ser reutilizado?
3. ¿Cómo garantizas que ningún servicio nuevo se implemente **a medias** (sin calcular su factura)?

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Este reto lo resuelves tú, aplicando **TODO** lo aprendido:

1. Diseña la clase abstracta **ServicioPublico.java** con los atributos `protected` y el método abstracto `calcularFactura()`. El método `emitirRecibo()` debe ser concreto y usar `calcularFactura()` internamente.
2. Diseña las tres hijas: **Agua.java**, **Luz.java**, **Gas.java**. Cada una debe recibir su consumo y aplicar su fórmula.
3. Diseña un **MainServicios.java** que:
 - Cree una factura de agua con 15 m³ de consumo
 - Cree una factura de luz con 250 kWh
 - Cree una factura de gas con 12 m³
 - Emita los tres recibos

Entrega: los cuatro archivos .java funcionando. Intenta también crear un objeto `new ServicioPublico(...)` y observa el error del compilador. Anótalo.

Cierre metacognitivo del taller

REFLEXIÓN METACOGNITIVA — ¿QUÉ APRENDÍ?

Antes de terminar, tómate 5 minutos para responder en tu cuaderno:

1. Con tus propias palabras, ¿qué es la abstracción y en qué se diferencia de la herencia simple?
2. ¿Por qué no se puede escribir *new Figura()* pero sí *new Circulo()*?
3. ¿En qué ejercicio te resultó **más claro** el concepto de "contrato obligatorio"?
4. Menciona **un ejemplo de la vida real** donde exista una abstracción (algo que define qué se hace, pero cada implementación lo hace de forma distinta).
5. Si tuvieras que explicar la diferencia entre *abstract class* y una clase normal, ¿qué dirías?
6. ¿Qué duda te quedó? Anótala para consultarla con tu instructor.

Conclusión del taller

La abstracción te permitió **concentrarte en lo esencial** y posponer los detalles hasta el último momento. Mediante clases y métodos abstractos definiste **contratos obligatorios** que cualquier subclase debe cumplir, asegurando un diseño sólido. Junto con la herencia, abre el camino directo al último pilar de la POO: el **polimorfismo**, que verás en el próximo taller. Recuerda: **una clase abstracta no se instancia**; solo sirve como base para otras clases concretas.

SENA - Servicio Nacional de Aprendizaje
Formación Profesional Integral
Programa: Análisis y Desarrollo de Software